C++ Data Structures for Low Latency Systems

Zhuo Zhao joezhao@bu.edu

Module 2: Contiguous Memory Structures

std::array, std::vector, Cache Locality, and Memory Layout

Lecture Notes

SAMPLE

1. Memory Model Recap: Why It Matters for Latency

Before diving into specific containers, we need to understand where data lives and how the CPU actually accesses it. In a low-latency context, understanding the hardware is not optional—it's the foundation of every data structure decision.

1.1 Stack vs Heap

In C++, variables can live in two main regions of memory:

- **Stack:** Fast allocation (just move the stack pointer), automatic cleanup, limited size (typically 1–8 MB). Local variables, function arguments, and small fixed-size data go here.
- **Heap:** Dynamic allocation via new/malloc, flexible size, but involves system calls and potential fragmentation. Containers like std::vector store their data on the heap.

> ⚡ **In HFT, every heap allocation on the hot path is a potential latency spike. This is why we pre-allocate.**

1.2 CPU Cache Hierarchy

Modern CPUs don't read directly from main memory. Instead, data flows through a cache hierarchy:

Cache Level	Typical Size	Latency (approx.)	Notes
L1	32–64 KB	`~1 ns (3-4 cycles)`	Per-core, split I-cache / D-cache
L2	256 KB–1 MB	~3–7 ns	Per-core
L3	8–64 MB	~10–20 ns	Shared across cores
Main Memory	GBs	~50–100 ns	DRAM, orders of magnitude slower

The CPU loads data in cache lines (typically 64 bytes). When you access one byte, the entire 64-byte cache line is fetched. This means if your next piece of data is nearby in memory, it's likely already in cache—this is the essence of cache locality.

> ⚡ **A cache miss to main memory costs ~50–100x more than an L1 hit. In HFT, this is the difference between a fill and a miss on a trade.**

1.3 What is a Cache Line?

A cache line is the smallest unit of data transfer between main memory and CPU cache. On x86-64, it's 64 bytes.

SAMPLE

```
// Visualizing a 64-byte cache line:
// |-------- 64 bytes --------|
// | int[0] | int[1] | ... | int[15] |   <- 16 ints fit in one cache line
//
// Accessing int[0] loads int[1]..int[15] for free!
```

This is why contiguous memory is king: iterating through a vector means each cache line fetch gives you 16 integers for the price of one memory access.

SAMPLE

2. std::array: Fixed-Size, Stack-Allocated

2.1 What is std::array?

std::array<T, N> is a fixed-size container that wraps a C-style array with the benefits of the STL interface. It was introduced in C++11.

```
#include <array>

std::array<double, 5> prices = {100.25, 100.30, 100.28, 100.32, 100.35};

// Access
double mid = prices[2];          // No bounds checking
double safe = prices.at(2);      // Throws std::out_of_range

// Size is compile-time constant
constexpr size_t n = prices.size();  // 5, known at compile time

// Iteration (range-based for works)
for (const auto& p : prices) { /* ... */ }
```

2.2 Key Properties

Property	std::array<T,N>	C-style T arr[N]
Storage	`Stack (inline)`	`Stack (inline)`
Size known?	`Yes (.size())`	Decays to pointer
Bounds check	`.at() available`	No
STL compatible	`Yes (iterators, algorithms)`	Partially
Copy semantics	`Value copy (deep)`	Not directly copyable
Zero overhead	`Yes (same as raw array)`	Yes

2.3 When to Use std::array in HFT

- **Fixed-size order books:** When you know the number of price levels at compile time (e.g., top-of-book with 10 levels)
- **Lookup tables:** Symbol-to-index mappings with a known universe
- **Small buffers:** Timestamp arrays, rolling windows with fixed width
- **Hot-path data:** Any small, fixed-size data that must avoid heap allocation

> ⚡ **std::array lives on the stack = zero allocation overhead = predictable latency. Use it when size is known at compile time.**

SAMPLE

3. std::vector: The HFT Workhorse

3.1 Internal Structure

std::vector<T> is a dynamically-sized array. Internally, it maintains three pointers:

```
// Simplified internal structure of std::vector<T>:
struct vector {
    T* _begin;     // pointer to first element
    T* _end;       // pointer past last element (size = _end - _begin)
    T* _capacity;  // pointer past allocated storage
};
// sizeof(vector) = 3 pointers = 24 bytes on 64-bit
// Actual data lives on the heap, contiguously
```

The data is stored in a single contiguous block on the heap. This is what makes vector so cache-friendly: iterating through a vector is essentially iterating through a raw array.

3.2 Growth Strategy: The Hidden Cost

When a vector runs out of capacity, it must allocate a new larger block, copy all elements, and free the old one. Most implementations grow by a factor of 2x (GCC) or 1.5x (MSVC).

```
std::vector<int> v;
// v.size()=0, v.capacity()=0

v.push_back(1);  // allocate capacity=1, copy 0 elements
v.push_back(2);  // allocate capacity=2, copy 1 element
v.push_back(3);  // allocate capacity=4, copy 2 elements
v.push_back(4);  // no reallocation! capacity=4
v.push_back(5);  // allocate capacity=8, copy 4 elements
// ...
// Each reallocation: O(n) copy + heap allocation + potential cache
invalidation
```

> ⚡ **In HFT, a single reallocation on the hot path can cause a latency spike of microseconds. This is why reserve() is non-negotiable.**

3.3 reserve() and Pre-allocation

The reserve() function pre-allocates memory without changing the size of the vector. This is the single most important vector optimization in low-latency code.

```
// BAD: Repeated allocations on hot path
std::vector<Order> orders;
for (const auto& msg : market_data) {
```

```
    orders.push_back(parse_order(msg));  // may reallocate!
}

// GOOD: Pre-allocate during initialization
std::vector<Order> orders;
orders.reserve(MAX_ORDERS_PER_DAY);  // one allocation upfront
// Now push_back will never reallocate (up to MAX_ORDERS_PER_DAY)
```

Reserve Strategy Patterns

Pattern	Code	Use Case
Known max size	`v.reserve(N)`	Order queues, message buffers
Estimate + headroom	`v.reserve(estimate * 1.5)`	Variable-size daily data
Reuse across cycles	`v.clear() (keeps capacity)`	Per-tick processing buffers
Shrink if needed	`v.shrink_to_fit()`	`End-of-day cleanup (never on hot path)`

3.4 Common Operations and Their Costs

Operation	Time Complexity	Latency Impact	Notes
`push_back (no realloc)`	`O(1) amortized`	Very low	Just write + increment pointer
`push_back (realloc)`	`O(n)`	HIGH	Heap alloc + copy all elements
operator[]	`O(1)`	Very low	Pointer arithmetic, no bounds check
`at()`	`O(1)`	Low	Bounds check adds a branch
`insert (middle)`	`O(n)`	HIGH	Must shift all subsequent elements
`erase (middle)`	`O(n)`	HIGH	Must shift all subsequent elements
`erase (back)`	`O(1)`	Very low	Just decrement end pointer
`clear()`	`O(n) or O(1)`	Low	Destroys elements, keeps capacity
iteration	`O(n)`	Very low per elem	Sequential = cache-friendly

> ⚡ **Prefer push_back/pop_back (O(1)) over insert/erase in the middle (O(n)). If you need frequent middle operations, you may want a different data structure.**

3.5 Practical Tips for HFT

- **Always reserve() during initialization,** never on the hot path
- **Use clear() instead of creating new vectors** — it preserves the allocated memory
- **Prefer emplace_back over push_back** to avoid unnecessary copies (constructs in-place)
- **Use data() to get raw pointer** when interfacing with C APIs or for SIMD operations
- **Avoid vector<bool>** — it's a special case that uses bitwise packing, breaking pointer semantics and cache predictability. Use vector<char> or std::bitset instead.

SAMPLE

4. Cache Locality: Why Contiguous Memory Wins

4.1 Sequential vs Random Access

Let's compare iterating through a vector (contiguous) vs. a linked list (scattered pointers):

```
// Vector: contiguous memory
// Memory: [A][B][C][D][E][F][G][H]
// Cache:  |---cache line 1---|---cache line 2---|
// Access A -> B,C,D,E,F,G,H already in cache!

// Linked list: pointer chasing
// Memory: [A]-->[?????]-->[B]-->[?????]-->[C]-->...
// Each -> is a pointer dereference = potential cache miss
// Each node could be ANYWHERE in memory
```

4.2 The Numbers Don't Lie

Classic benchmark: iterate and sum N integers.

Container	N = 1K	N = 100K	N = 10M	Cache Behavior
std::vector	~0.3 μs	~30 μs	~3 ms	Sequential prefetch
std::list	~2 μs	~800 μs	~120 ms	Random pointer chase
std::deque	~0.5 μs	~50 μs	~5 ms	Chunked sequential

The linked list is 30–40x slower for large N, and the gap grows with data size. This is almost entirely due to cache misses.

> ⚡ **Bjarne Stroustrup (the creator of C++): "Vector is almost always the right choice. Even when you think you need a linked list, measure first."**

4.3 Prefetching and Predictability

Modern CPUs have hardware prefetchers that detect sequential access patterns and load the next cache lines before you need them. Contiguous memory enables this; random access defeats it.

- With a vector, the CPU can **predict** your next access and have data ready before you ask
- With a linked list, every **next** pointer is a surprise — the prefetcher can't help you

- This is why even O(n) operations on a vector (like linear search) often beat O(log n) operations on a tree **for small to medium N**

4.4 Implications for HFT Data Structures

- **Flat arrays beat trees:** A sorted vector with binary search often outperforms std::map (red-black tree) for read-heavy workloads
- **Pool allocators help:** When you must use node-based structures, allocating nodes from a contiguous pool restores some cache locality
- **Measure everything:** perf stat can show you L1/L2/L3 cache miss rates — this is the ground truth

SAMPLE

5. SOA vs AOS: Memory Layout for Performance

5.1 The Problem

When you have a collection of objects with multiple fields, there are two fundamental ways to lay them out in memory:

AOS: Array of Structures (the default in C++)

```
// AOS: Each struct is contiguous, array of structs
struct Quote {
    uint64_t timestamp;  // 8 bytes
    double   bid;        // 8 bytes
    double   ask;        // 8 bytes
    uint32_t bid_size;   // 4 bytes
    uint32_t ask_size;   // 4 bytes
    char     symbol[8];  // 8 bytes
};  // Total: 40 bytes per struct

std::vector<Quote> quotes;  // [Q0|Q1|Q2|Q3|...]
// Memory: [ts0|bid0|ask0|bsz0|asz0|sym0|ts1|bid1|ask1|...]
```

SOA: Structure of Arrays

```
// SOA: Each field in its own array
struct QuoteSOA {
    std::vector<uint64_t> timestamps;
    std::vector<double>   bids;
    std::vector<double>   asks;
    std::vector<uint32_t> bid_sizes;
    std::vector<uint32_t> ask_sizes;
    // ...
};
// Memory: [ts0|ts1|ts2|...][bid0|bid1|bid2|...][ask0|ask1|ask2|...]
```

5.2 When SOA Wins

If your hot loop only touches a few fields, SOA packs more useful data per cache line:

```
// Task: compute mid-price for all quotes

// AOS: loads 40 bytes per quote, but only uses bid + ask (16 bytes)
// Wasting 60% of each cache line on unused fields!
for (const auto& q : quotes) {
    double mid = (q.bid + q.ask) / 2.0;
```

```
}

// SOA: loads ONLY bid[] and ask[] arrays
// Every byte in cache is useful data!
for (size_t i = 0; i < n; ++i) {
    double mid = (soa.bids[i] + soa.asks[i]) / 2.0;
}
```

Metric	AOS	SOA
Bytes loaded per element	`40 (full struct)`	`16 (bid + ask only)`
Cache utilization	40%	100%
Elements per cache line	1.6	`4 (doubles)`
SIMD friendly?	`No (mixed types)`	`Yes (homogeneous arrays)`

5.3 When AOS Wins

AOS is better when you typically access all (or most) fields of each object together:

- **Processing individual orders:** If you read timestamp, price, quantity, and symbol for each order, AOS keeps all fields adjacent
- **Passing objects to functions:** AOS lets you pass a single reference instead of indices into multiple arrays
- **Simpler code:** AOS is natural in C++ and easier to maintain

5.4 Hybrid Approach: Hot/Cold Split

In practice, HFT systems often use a hybrid: group frequently-accessed fields together and separate rarely-used fields:

```
// Hybrid: Hot data contiguous, cold data separate
struct QuoteHot {
    double bid;
    double ask;
    uint64_t timestamp;
};  // 24 bytes - fits more per cache line

struct QuoteCold {
    uint32_t bid_size;
    uint32_t ask_size;
    char symbol[8];
    // ... other metadata
```

SAMPLE

```
};

std::vector<QuoteHot>  hot_data;   // hot path iterates this
std::vector<QuoteCold> cold_data;  // only accessed when needed
// Invariant: hot_data[i] corresponds to cold_data[i]
```

⚡ **The Hot/Cold split is one of the most practical optimizations in HFT: keep your hot-path data small and contiguous, push everything else out of the critical cache lines.**

SAMPLE

6. Practical Exercises

Exercise 1: Measure the Cost of Reallocation

Write a benchmark that compares two approaches:

- Approach A: push_back 1,000,000 integers into a vector WITHOUT calling reserve()
- Approach B: push_back 1,000,000 integers into a vector WITH reserve(1000000)

Use <chrono> to measure wall time. Print the number of reallocations in Approach A (hint: track capacity changes).

Exercise 2: Vector vs List Cache Test

Create both a std::vector<int> and a std::list<int> with 1,000,000 elements. Measure the time to sum all elements in each. Run each test 100 times and report the average. What speedup do you see? Can you explain it in terms of cache misses?

Exercise 3: SOA vs AOS Benchmark

Implement the Quote structure in both AOS and SOA layouts as shown in Section 5. Populate with 10 million entries. Benchmark two operations:

- Operation 1 (SOA-friendly): Compute the average bid-ask spread across all quotes
- Operation 2 (AOS-friendly): Print all fields of the quote with the highest bid price

Verify that SOA wins on Operation 1 and AOS wins (or ties) on Operation 2.

Exercise 4: Build a Pre-allocated Order Buffer

Design an OrderBuffer class that:

- Pre-allocates space for N orders at construction time
- Supports add_order() and clear_for_next_tick() operations
- Never allocates memory after construction
- Uses clear() to reset between ticks while keeping capacity

Write a stress test that simulates 10,000 ticks with random order counts (1–100 per tick) and verify zero allocations after the first tick.

SAMPLE

7. Key Takeaways

Decision Framework

Question	Answer	Recommendation
Size known at compile time?	Yes	std::array
Size known at compile time?	No	`std::vector + reserve()`
Need to avoid ALL heap alloc?	Yes	std::array or stack buffer
Hot loop touches few fields?	Yes	Consider SOA or hot/cold split
Hot loop touches all fields?	Yes	`AOS (default struct layout)`
Need middle insert/erase?	Rarely	`Still use vector (shift cost < cache miss cost)`
Need middle insert/erase?	Frequently	`Consider other structures (Module 4-5)`

Golden Rules for Low-Latency C++

- **Rule 1:** Contiguous memory first. std::vector is your default container.
- **Rule 2:** Pre-allocate everything. reserve() at initialization, never on the hot path.
- **Rule 3:** Avoid pointer chasing. Every indirection is a potential cache miss.
- **Rule 4:** Think in cache lines (64 bytes). Keep hot data small and packed.
- **Rule 5:** Measure, don't guess. Use perf stat, Google Benchmark, and real data.

> ⚡ **The #1 performance optimization in C++ is not clever algorithms—it's data layout. Design your memory first, then write code around it.**

Next: Module 3 — Associative Containers & Hash Maps for HFT

SAMPLE